CISC2006: Algorithm Design and Analysis
Homework 1 **Due Date:** 23:59, 14 Feb 2025

Rules: Discussion of the problems is permitted, but writing the assignment together is not (i.e. you are not allowed to see the actual pages of another student). You can get at most 100 points if attempting all problems. Please make your answers precise and concise.

1. (10 pts) Give "True" or "False" for each of the following statements.

   No proof is needed.

   - $T(n) = 3n^2 + 5n \cdot \log_2 n = O(n)$.
   - $T(n) = 4^{\log_2 n} + \sqrt{n} = \Omega(n^2)$.
   - $T(n) = 3n^2 + 9n = O(n^3)$.
   - $T(n) = 4 \cdot (\log_2 n)^5 + 5\sqrt{n} + 10 = \Theta(\sqrt{n})$.
   - $T(n) = (\log_2 n)^{\log_2 n} + n^4 = \Theta(n^4)$.

   Solution:

   - **False**
   - **True**
   - **True**
   - **True**
   - **False**

2. (10 pts) Prove that function $T(n) = \sum_{i=1}^{n} \log i = \Theta(n \log n)$.

   Solution:

   For all $n \geq 10$, we have $\sum_{i=1}^{n} \log i \leq \sum_{i=1}^{n} \log n = n \log n$, which (by definition of big O notation) implies that $T(n) = O(n \log n)$.

   For all $n \geq 10$, we have $\sum_{i=1}^{n} \log i \geq \sum_{i=n/2}^{n} \log i \geq (n/2) \log(n/2) \geq \frac{1}{4} \cdot n \log n$, which (by definition of Ω notation) implies that $T(n) = \Omega(n \log n)$.

   Combining the above two, we have that $T(n) = \Theta(n \log n)$.

3. (10 pts) Given a sequence of $A = \{a_1, a_2, \ldots, a_n\}$ of n integers, where $a_1 \leq a_2 \leq \ldots \leq a_n$ and another integer K, give an algorithm that outputs

- three different $i, j, k \in \{1, 2, \ldots, n\}$ such that $a_i + a_j + a_k = K$, or
- "do not exist" if they don't exist.

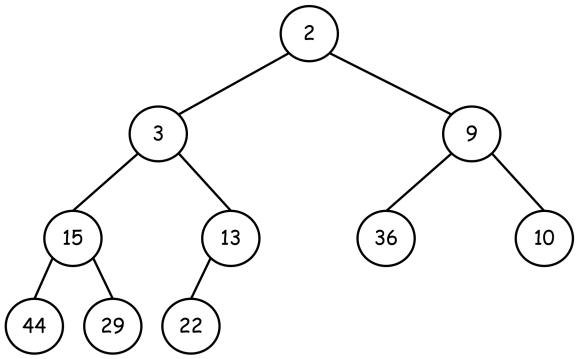
Complete the missing steps in the pseudo-code of the algorithm.

Algorithm 1: Sum_of_three(A, K)

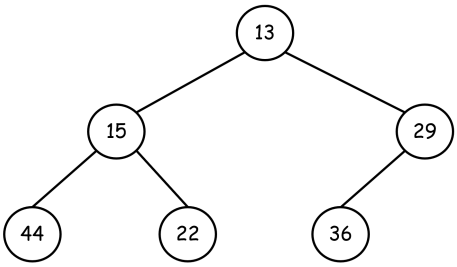
```
let n ← |A| and assume A = {a_1, a_2, ..., a_n}.
for i = 1, 2, ..., n − 2 do
    j ← i + 1 and k ← n.
    while j < k do
        if a_i + a_j + a_k = K then
            Output: (i, j, k).
        else if a_i + a_j + a_k < K then
            j ← j + 1.
        else if a_i + a_j + a_k > K then
            k ← k − 1.
Output: "do not exist".
```

4. (20 pts) Draw the heap (as a binary tree) after executing the following operations on an initially empty heap (you do not need to show the intermediate steps):

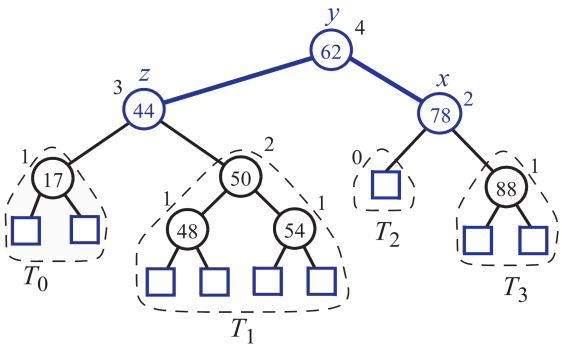
   - insertions of elements $22, 15, 36, 44, 10, 3, 9, 13, 29, 2$ (inserted one-by-one);
     Solution:



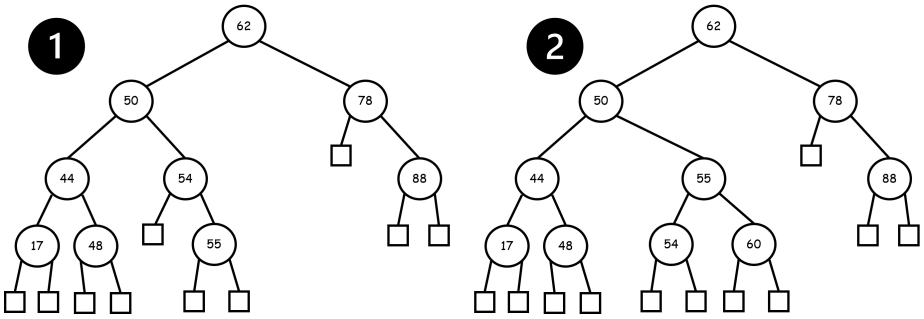
   - removeMin() four times.
     Solution:



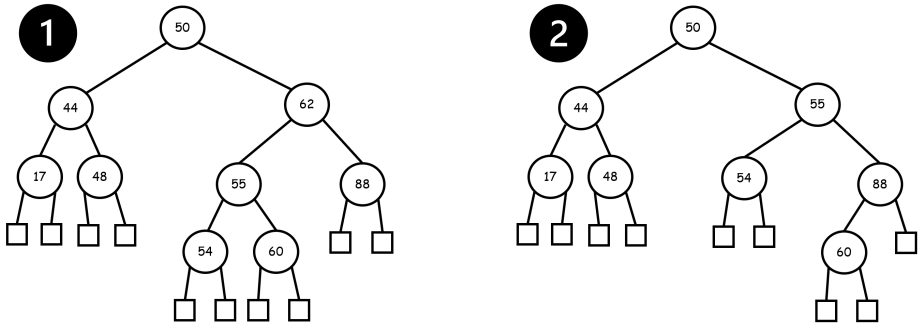
5. (20 pts) Draw the AVL tree after executing each of the following operations on:



- insertion of element 55, then insertion of element 60;
  Solution:



- removal of element 78, then removal of element 62.
  Solution:



6. (20 pts) **Different Implementations of Priority Queue**.

Implement a priority queue data structure class to store a collection of different numbers that supports the following operations:

- insert(e) : insert an element e into the priority queue;
- min() : return the minimum element in the priority queue;
- removeMin() : remove the minimum element in the priority queue;
- size() : return the total number of elements in the priority queue;
- isEmpty() : return True if the priority queue is empty; False otherwise.
- printPQ() : list all elements in the priority queue. For a heap, list the elements from top-level to bottom-level, and for each level from left to right.

Use the following data structures for three different implementations:

- Unsorted Doubly Linked List;
- Sorted Doubly Linked List;
- Heap (implemented using an array).

In the main function, we read an array $A = \{a_1, a_2, \ldots, a_n\}$ of n different integers, and use the three different implementations of priority queue to do sorting.

In particular, we do the following for each version of priority queues.

We first initialize a priority queue, which is empty. Then we insert the numbers in A one-by-one, and sort the numbers into another array $B = \{b_1, b_2, \ldots, b_n\}$ for output by repeatedly calling min() and removeMin().

7. (30 pts) **Different Implementations of Binary Search Tree**.

Implement a binary search tree (BST) data structure class to store a collection of different numbers that supports that following operations

- insert(e) : inset element e into the BST;
- find(k) : return the pointer that points to an element with key $= k$; if no such element exists, return Null;
- remove(k) : remove the element with key $= k$ if such element exists;
- remove(p) : remove the element pointed by pointer p;
- size() : return the total number of elements in the BST;
- isEmpty() : return True if the BST is empty; False otherwise.
- printTree() : print the whole BST (use indentation to show the structure).

Use the following data structures for the two different implementations:

- Binary tree without height balance property;
- AVL tree.

In the main function, we will read an array $A = \{a_1, a_2, \ldots, a_n\}$ of n different integers, and insert the numbers into the two different implementations of BST one-by-one. Then we read another array $B \subseteq A$ of integers, each of which appeared in A, and remove the integers in B from the BST.

Finally, we output the resulting BST using printTree().